

Git Mastery Guide

Concepts + Interview Prep + Azure DevOps & Terraform Strategy

Beginner | Mid-Level | Senior / DevOps Engineer

Designed for Students & Interview Aspirants

Section 1: Git Kya Hai Aur Kyun Seekhna Chahiye?

Socho ek team 5 developers ki hai — sab ek hi project pe kaam kar rahe hain. Agar koi version control nahi hoga, toh kya hoga? Ek banda file overwrite karega, doosra purani code se kaam karega — total chaos!

Git ek distributed version control system hai jo:

- Tumhara code history safe rakhta hai
- Multiple logon ko ek saath kaam karne deta hai
- Galti ho toh purani state mein wapas jaane deta hai
- Teams mein code review, collaboration aur deployment automate karta hai



Analogy

Git ek time machine ki tarah hai apne code ke liye. Har commit ek save point hai — jab chahein wapas jao!

Section 2: Core Concepts (Fundamentals)

2.1 Repository (Repo)

Repository ek folder hai jahan Git tumhara sara code aur uska history track karta hai. Do types hote hain:

Type	Description
Local Repository	Tumhare apne computer pe hota hai. git init se banta hai.
Remote Repository	Server pe hota hai — GitHub, Azure Repos, GitLab etc. Team ke sab log yahan se sync karte hain.

```
git init          # Naya local repo banao
git clone <url>    # Remote repo apne machine pe lao
```

2.2 Working Tree, Staging Area & Repository

Git mein teen jagah hoti hain jahan tumhari files rehti hain:

Zone	Matlab Kya Hai
Working Tree (Working Dir)	Jahan tum files edit karte ho — tumhara normal folder
Staging Area (Index)	Ek in-between zone — 'ye wali changes commit karni hain' — git add se yahan aati hain
Repository (.git folder)	Permanent history — git commit se yahan jaata hai



Trick

Socho ghar shift kar rahe ho: Working Tree = saman bikhra hua, Staging = jo boxes pack ho gaye, Repository = jo truck mein load ho gaya!

```
git add file.tf    # Staging mein dalo
git add .          # Sab files staging mein
git commit -m 'message' # Repository mein save
```

2.3 Commit

Commit ek snapshot hai tumhare code ka ek specific point pe. Har commit ka ek unique SHA hash hota hai (jaise: a3f8b2c).

- Ek commit mein sirf ek logical change honi chahiye
- Commit message clear aur descriptive hona chahiye
- feat: add storage account Terraform module — yeh ek acha message hai

```
git commit -m 'feat: add AKS node pool config'
git log --oneline          # Saari commits dekho
git show <commit-hash>    # Specific commit ka detail
```

2.4 Branch

Branch ek alag line of development hai. Jaise ek highway pe alag exit lena — main road (main/master) pe traffic affect nahi hoti.

Branch Type	Purpose
main / master	Production-ready code — yahan directly push nahi karte
feature/xyz	Naya feature develop karna
fix/bug-123	Bug fix karna
release/v1.2	Release preparation
hotfix/critical	Production mein urgent fix

```
git branch feature/add-vnet    # Branch banao
git checkout feature/add-vnet  # Us branch pe jao
git checkout -b fix/nsg-rule    # Banao + switch ek command mein
git branch -a                  # Saari branches dekho
```

2.5 Merge vs Rebase

Dono cheezein ek branch ki changes doosri mein lene ke liye use hoti hain — lekin tarika alag hai:

Command	Kya Hota Hai
git merge	Dono branches ki history maintain hoti hai. Ek extra 'merge commit' banta hai. Team mein safe hai.
git rebase	Branch ki history clean aur linear dikhti hai. History rewrite hoti hai — shared branches pe kabhi nahi!



Rule

Shared/public branch (main, develop) pe kabhi rebase mat karo — dusron ka history corrupt ho jaata hai!

```
git merge feature/add-vnet    # Merge karo
git rebase main                # Current branch ko main ke upar rakh do (local only!)
git rebase -i HEAD~3          # Interactive rebase — last 3 commits clean karo
```

2.6 Remote Operations

Command	Kya Karta Hai
<code>git fetch</code>	Remote se naye commits lao — apni local branch merge mat karo
<code>git pull</code>	fetch + merge ek saath (ya fetch + rebase agar configured ho)
<code>git push</code>	Apni local commits remote pe bhejo
<code>git remote -v</code>	Kaunse remote servers configured hain

```
git fetch origin          # Remote dekho
git pull origin main      # Latest code lao
git push origin feature/add-vnet # Apni branch push karo
git push -u origin feature/xyz  # Push + upstream set karo
```

2.7 Git Stash

Kaam beech mein chodna pad jaaye (emergency fix!) toh stash karo — changes temporarily save ho jaate hain:

```
git stash          # Uncommitted changes save karo
git stash list     # Saare stashes dekho
git stash pop      # Last stash wapas lao + delete karo
git stash apply stash@{1} # Specific stash apply karo (delete nahi hoga)
```

2.8 Git Tags

Tags specific commits ko mark karte hain — usually releases ke liye:

```
git tag v1.0.0          # Lightweight tag
git tag -a v1.0.0 -m 'First release' # Annotated tag (recommended)
git push origin v1.0.0  # Tag push karo
git push origin --tags  # Saare tags push karo
```

2.9 Important Git Internals

Ye concepts senior-level interviews mein pooche jaate hain:

Concept	Explanation
HEAD	Tumhara current position — jis commit pe tum ho. Detached HEAD = direct commit pe ho, kisi branch pe nahi.
SHA Hash	Har commit, tree, blob ka unique 40-char identifier. Immutable history ka basis.
Blob	Git object jo file content store karta hai (filename nahi)

Concept	Explanation
Tree	Git object jo directory structure store karta hai
.git/refs	Branches aur tags ke pointers yahan hote hain
git reflog	Tumhari saari local actions ka log — galti ho toh rescue tool!

```
git reflog          # Sab kuch recover karo
git reset --hard HEAD~1  # Last commit undo (local only)
git revert <commit-hash> # Purani commit undo karo – history safe
```

Section 3: Git Branching Strategies

Branching strategy decide karti hai ki team kaise collaborate karegi, kab deploy hoga aur code kaise review hoga. Azure DevOps pipelines ke saath yeh aur bhi important ho jaata hai.

3.1 GitFlow

Classic enterprise strategy — structured aur formal. Bade projects ke liye achha hai.

Branch	Role
main	Production code — kabhi direct push nahi
develop	Integration branch — sab features yahan merge hote hain
feature/*	Ek feature ke liye — develop se branch, develop mein merge
release/*	Release preparation — bugfixes, version bump
hotfix/*	Production mein urgent fix — main aur develop dono mein merge

Azure DevOps Tip

GitFlow ke saath Azure Pipelines mein branch filters use karo: develop pe CI, release/* pe staging deploy, main pe production deploy.

3.2 Trunk-Based Development (TBD)

Modern teams aur DevOps culture ke liye — fast delivery, small commits:

- Sab developers directly main/trunk pe ya short-lived feature branches pe kaam karte hain
- Branches 1-2 din se zyada nahi chalti
- Feature Flags use karte hain incomplete features ko hide karne ke liye
- CI/CD bahut tight hota hai — har commit pe build + test

TBD + Terraform

Terraform changes bhi daily main mein merge hote hain. Terraform Cloud ya Azure DevOps pipeline har PR pe plan run karta hai, merge pe apply.

3.3 GitHub Flow (Simplified)

Simple projects aur small teams ke liye — main + feature branches only:

1. main se feature branch banao
2. Kaam karo, commit karo
3. Pull Request kholo
4. Code review + CI pass
5. Merge to main

6. Deploy immediately

Azure Repos Tip

Azure Repos mein Branch Policies set karo: minimum 1 reviewer, linked work item, build validation (CI must pass before merge).

Section 4: Git in Azure DevOps & Terraform Workflows

Real-world DevOps mein Git aur Azure DevOps/Terraform ek saath kaam karte hain. Yeh section us context ko cover karta hai.

4.1 Azure Repos Setup

- Azure Repos = Git-based VCS inside Azure DevOps
- HTTPS ya SSH se clone karo
- Branch Policies: Require PR, reviewers, linked work items, build validation
- Protected branches: main, develop — direct push block

```
git remote add origin https://dev.azure.com/org/project/_git/repo
git push -u origin main
```

4.2 Terraform Monorepo Structure (Best Practice)

Ek common pattern — sab infra code ek repo mein, environment-wise folders:

Path	Purpose
<code>infra/modules/</code>	Reusable Terraform modules (vnet, aks, storage, etc.)
<code>infra/envs/dev/</code>	Dev environment variables aur backend config
<code>infra/envs/staging/</code>	Staging environment
<code>infra/envs/prod/</code>	Production — strict access, PR required
<code>.gitignore</code>	<code>.terraform/</code> , <code>*.tfstate</code> , <code>*.tfvars.json</code> ko ignore karo
<code>.pre-commit-config.yaml</code>	terraform fmt, tflint, tfsec run karo commit se pehle

```
# .gitignore for Terraform repo
.terraform/
*.tfstate
*.tfstate.backup
*.tfvars          # Secrets kabhi commit mat karo!
override.tf
```

4.3 CI/CD Pipeline with Git Triggers

Azure Pipelines mein Git events pe trigger hote hain:

Git Event	Pipeline Action
PR to main	terraform plan run hota hai — plan output PR pe comment ata hai
Merge to main	terraform apply run hota hai — infra deploy hoti hai
Tag push (v*.*.*)	Production release pipeline trigger hota hai
Feature branch push	Unit tests, terraform validate, tfint run karte hain

```
# azure-pipelines.yml example
trigger:
  branches:
    include:
      - main
  paths:
    include:
      - infra/

pr:
  branches:
    include:
      - main
```

4.4 Git Security Best Practices (Production)

Security Area	Best Practice
Secrets in Git	KABHI NAHI — Azure Key Vault use karo. git-secrets ya pre-commit hooks se scan karo.
Signed Commits	gpg --sign — production repos mein enforce karo
CODEOWNERS	Critical files ke liye specific team ka review mandatory
Branch Protection	Force push disable, PR required, status checks required
Audit Logs	Azure DevOps audit logs enable karo — who pushed what, when

Section 5: Interview Questions — Teeno Levels

Yeh questions aajkal sabse zyada pooche jaate hain — Beginner se leke Senior DevOps tak. Har question ke saath level marked hai.

5.1 Git Fundamentals Questions

Interview Question	Key Answer Points	Level
Git aur GitHub mein kya difference hai?	Git = VCS tool (local). GitHub/Azure Repos = hosting platform (remote + collaboration features).	Beginner
git pull aur git fetch mein kya difference hai?	fetch = remote history lao, merge mat karo. pull = fetch + merge. fetch zyada safe hai.	Beginner
Staging area kyun use karte hain directly commit kyun nahi?	Staging se selective commits bante hain — ek hi baar mein sab nahi, logical grouping hoti hai.	Beginner
git stash kab use karte hain?	Jab kaam beech mein chodna ho bina commit kiye — emergency branch switch ya urgent hotfix.	Beginner
HEAD kya hota hai Git mein?	Current branch ka latest commit pointer. Detached HEAD = kisi branch pe nahi, directly commit pe ho.	Beginner
git merge aur git rebase mein kya difference hai?	Merge = history preserve, extra commit banta hai. Rebase = linear history, history rewrite. Shared branches pe rebase kabhi nahi.	Mid-Level
git cherry-pick kya hai aur kab use karte hain?	Ek specific commit ko doosri branch pe apply karna. Hotfix ko main se develop mein lena — common use case.	Mid-Level
Fast-forward merge kab hota hai?	Jab target branch mein koi naya commit nahi hota — simply pointer aage move ho jaata hai, no merge commit.	Mid-Level
git reset --soft, --mixed, --hard mein kya fark hai?	soft = commit undo, changes staged. mixed = commit undo, changes unstaged. hard = sab delete (khatra!).	Mid-Level
git revert vs git reset — production mein kaunsa use karein?	git revert — history safe rakhta hai. reset shared history change karta hai — production mein kabhi nahi.	Mid-Level
Monorepo vs Polyrepo — kab kya use karein? Trade-offs kya hain?	Monorepo = ek repo, easy cross-service changes, bade CI/CD.	Senior

Interview Question	Key Answer Points	Level
	Polyrepo = isolation, independent deploy. Team size aur coupling pe depend karta hai.	
Git objects internals explain karo — blob, tree, commit, tag.	Blob = file content. Tree = directory structure. Commit = tree + parent + metadata. Tag = annotated pointer to commit. Sab SHA se identify hote hain.	Senior
Large binary files Git mein handle kaise karein?	Git LFS (Large File Storage) use karo. Binary files .gitattributes mein mark karo — pointers store hote hain, actual files separate storage mein.	Senior
Git submodules kya hai aur unke trade-offs?	Ek repo mein doosra repo embed karna. Versioned dependency. Downside: complex workflows, detached HEAD issues, team ko training chahiye.	Senior
Terraform repo mein .gitignore mein kya hona chahiye aur kyun?	.terraform/, *.tfstate, *.tfstate.backup, override.tf, *.tfvars (secrets!). State files sensitive data contain karte hain.	Senior

5.2 Branching Strategy Questions

Interview Question	Key Answer Points	Level
GitFlow aur Trunk-Based Development mein kya difference hai?	GitFlow = long-lived branches, multiple envs. TBD = short-lived branches, feature flags, fast delivery. Modern DevOps mein TBD preferred.	Mid-Level
Azure DevOps mein branch policies kaise set karte hain?	Repos > Branches > Branch Policies. Min reviewers, build validation, linked work items, comment resolution required set kar sakte hain.	Mid-Level
PR strategy kya hoti hai Terraform repos mein?	Every change PR se aaye, terraform plan output comment mein ho, atleast 1 reviewer, tfsec/tflint pass hona chahiye, protected branch.	Senior
Ek team mein multiple Terraform environments ke liye branching strategy design karo.	env-specific branches ya workspace strategy. main = prod, develop = staging. Per-env tfvars, separate state files, pipeline branch filters.	Senior
Hotfix ko GitFlow mein kaise handle karte hain?	main se hotfix/* branch banao, fix	Mid-Level

Interview Question	Key Answer Points	Level
	karo, main mein merge karo (tag karo), develop mein bhi merge karo taaki fix lost na ho.	

5.3 Azure DevOps Pipeline + Git Integration

Interview Question	Key Answer Points	Level
Azure Pipeline mein git trigger aur PR trigger mein kya difference hai?	trigger = branch pe merge/push hone pe chalta hai. pr: = PR open/update hone pe chalta hai (typically terraform plan).	Mid-Level
Pipeline mein Terraform state ko Git se alag kyun rakhte hain?	State mein sensitive data hota hai (passwords, IPs). Remote backend (Azure Storage) use karo, kabhi Git mein commit mat karo.	Mid-Level
OIDC/Workload Identity se Azure DevOps pipeline authenticate kaise hoti hai Git aur Terraform ke saath?	Service connection with federated credential. No secrets stored — Azure AD token on-the-fly milta hai. ARM_USE_OIDC=true set karo Terraform ke liye.	Senior
GitOps kya hai aur Terraform ke saath kaise implement karte hain?	Git single source of truth hai infra ke liye. Har change Git mein hoti hai, pipeline automatically apply karta hai. Drift detection bhi set karo.	Senior
Pipeline mein secrets Git mein commit ho jaayein toh kya karte hain?	Immediately rotate secrets. git filter-repo ya BFG se history clean karo. .gitignore update karo. Pre-commit hooks add karo future prevention ke liye.	Senior

Section 6: Quick Reference — Git Commands Cheatsheet

Setup & Init

```
git config --global user.name 'Apna Naam'
git config --global user.email 'email@example.com'
git config --global core.editor 'code --wait'
git init                                # Naya repo
git clone <url>                         # Copy remote repo
```

Daily Workflow

```
git status                             # Kya changed hai
git diff                               # Kya changes hain (unstaged)
git diff --staged                      # Staged changes dekho
git add <file>                         # Stage karo
git add -p                             # Interactive staging (partial files)
git commit -m 'message'                # Commit karo
git commit --amend                     # Last commit fix karo (push se pehle only)
```

Branches

```
git branch                             # Local branches
git branch -a                          # All branches (remote bhi)
git checkout -b feature/xyz            # Banao + switch
git switch -c feature/xyz              # Modern syntax
git branch -d feature/xyz              # Delete (merged only)
git branch -D feature/xyz              # Force delete
```

Merge & Rebase

```
git merge feature/xyz                  # Merge into current
git merge --no-ff feature/xyz          # Always merge commit banao
git merge --abort                       # Merge rok do
git rebase main                         # Rebase on main (local!)
git rebase -i HEAD~3                   # Interactive rebase
git cherry-pick <hash>                 # Specific commit lao
```

Undo & Recovery

```
git reset --soft HEAD~1                # Commit undo, changes staged
git reset --mixed HEAD~1               # Commit undo, changes unstaged
```

```
git reset --hard HEAD~1      # Commit + changes sab delete!
git revert <hash>             # Safe undo (new commit)
git restore <file>           # Working tree changes discard
git restore --staged <file>  # Unstage karo
git reflog                   # Sab actions ka log – rescue!
```

Remote

```
git remote -v                # Remote URLs
git fetch origin              # Remote history lao
git pull origin main          # Fetch + merge
git push origin branch-name  # Push karo
git push --force-with-lease   # Safe force push
git push origin --delete branch-name # Remote branch delete
```

Tags

```
git tag                      # List tags
git tag -a v1.0.0 -m 'msg'  # Annotated tag
git push origin v1.0.0      # Tag push
git push origin --tags      # Saare tags push
```

Stash

```
git stash                    # Save changes
git stash push -m 'wip: feature-xyz' # Named stash
git stash list               # Dekho
git stash pop                # Apply + delete
git stash apply stash@{0}    # Apply (keep stash)
git stash drop stash@{0}    # Delete specific
git stash clear              # Sab delete
```

Terraform + Azure DevOps Specific

```
# .gitignore for Terraform
.terraform/
*.tfstate
*.tfstate.backup
*.tfvars

# Pre-commit hooks (.pre-commit-config.yaml)
repos:
  - repo: https://github.com/antonbabenko/pre-commit-terraform
    hooks:
      - id: terraform_fmt
      - id: terraform_validate
```

```
- id: terraform_tflint  
- id: terraform_tfsec
```


Section 7: Interview Tips — Kaise Answer Karein?

Sirf command yaad karna kaafi nahi hai. Senior interviews mein 'why' aur 'trade-offs' pooche jaate hain.

7.1 STAR Format se Answer Karo

STAR Step	Git Interview Mein
Situation	Kaunsa project/team/problem tha?
Task	Tumhara kya kaam tha?
Action	Tumne Git mein kya kiya? Kaunsi strategy use ki?
Result	Kya hua? Deployment faster hua? Bugs km aye?

7.2 Common Mistakes Jo Avoid Karo

- Sirf commands bolna — why explain karo
- Rebase ko merge se better kehna bina context ke — dono ke use cases hain
- 'Hum GitHub use karte the' bolna Azure DevOps interview mein bina Azure Repos ka mention kiye
- Security ignore karna — production Git setup mein secrets management, CODEOWNERS, signed commits mention karo
- Terraform state aur Git mein confuse karna — state NEVER in Git!

7.3 Senior Level Pe Ye Zaroor Mention Karo

- GitOps principles (Flux, ArgoCD, ya simple pipeline-based GitOps)
- Branch protection policies + CODEOWNERS
- Monorepo vs polyrepo — trade-offs with Terraform modules
- git reflog for disaster recovery
- Pre-commit hooks for Terraform quality gates
- OIDC/Workload Identity for pipeline authentication — no stored secrets

Best of luck! Git master ban ja — ek commit at a time! 🚀

Concepts clear hain toh commands khud yaad ho jaate hain.